

WEB4 PHP - Game of Thrones Quotes

Nathalie Sonnleitner

Tim Peko

Inhaltsverzeichnis

1. Einleitung und Überblick	3
1.1. Nutzerfluss nach Rolle	3
1.2. Rollen und Berechtigungen	4
2. Datenmodell	5
2.1. ER-Diagramm	5
2.2. Normalisierung und Constraints	5
2.2.1. users	6
2.2.2. quotes	6
2.2.3. comments	6
2.2.4. quote_likes	6
2.2.5. comment_votes	7
2.3. Testdaten	7
3. Architektur	7
3.1. MVC	7
3.2. Komponentenübersicht	8
3.3. Verzeichnisstruktur	9
3.4. Ablauf: Login und Session	9
3.5. Ablauf: Kommentar anlegen (CRUD Create)	10
3.6. Ablauf: Kommentar löschen per REST	11
3.7. REST-Routing	12
4. Backend und Sicherheit	13
4.1. PDO und Prepared Statements	13
4.2. Validierung	13
4.3. Schutzmaßnahmen im Überblick	13
5. Frontend und Benutzeroberfläche	14
5.1. HTML, CSS und JavaScript	14
5.2. Foren- und Thread-Darstellung	14
5.3. Public Profile	15
5.4. Admin-Bereich	16
6. Testfälle	17
6.1. Automatisierter REST-Testlauf	17
6.1.1. Übersicht REST-Tests	17
6.1.2. Erwartete und beobachtete Werte	18
6.2. Manuelle UI-Tests	19
6.2.1. Übersicht manuelle Tests	19
6.2.2. Detailergebnisse manuelle Tests	19
7. Installation und Abgabe	21
7.1. Voraussetzungen	21
7.2. Installationsschritte	21
7.3. Datenbankzugang	21

7.4. Abgabeformat	21
-------------------------	----

1. Einleitung und Überblick

Diese Dokumentation beschreibt die Webanwendung **GoT Quotes**, ein Forum für berühmte Game-of-Thrones-Zitate, entwickelt im Rahmen der Übung WEB4 (Sommersemester 2026). Nutzer können Zitate lesen, kommentieren, bewerten und liken. Die Kommentare sind als verschachtelte Threads implementiert. Administratoren verwalten Zitate und Nutzerkonten und können bei Bedarf Bilder zu Zitaten hochladen.

Das Projekt setzt auf eine klare Umsetzung bewährter Webstandards: PHP 8.x, objektorientiertes MVC-Pattern, PDO für den Datenbankzugriff, konsequente Verwendung von Prepared Statements sowie serverseitiges Validierungskonzept. Alle Kernfunktionen wie das Kommentarsystem (CRUD), Benutzerverwaltung, Rollen (Gast, Nutzer, Admin) und sichere Formularverarbeitung sind vollständig umgesetzt. Besonders im Fokus stand Schutz vor SQL-Injection und Cross-Site-Scripting (XSS).

Start-URL: `http://localhost/` (nach Entpacken des ZIP-Archivs und Setzen des Apache-Document-Roots auf `public/`)

Projektmitglieder:

Name	Matrikelnummer
Nathalie Sonnleitner	s2510458003
Tim Peko	s2420458029

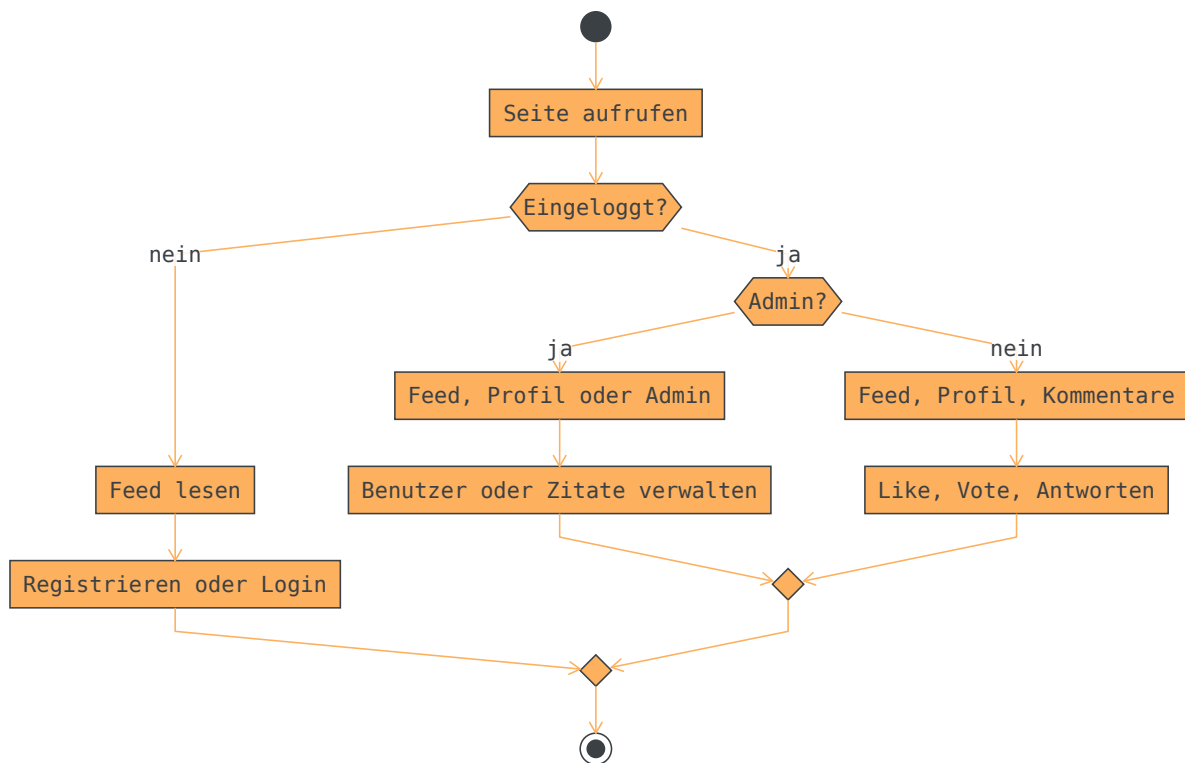
Wesentliche Projekt-Highlights:

- MVC-Struktur, saubere Trennung von Controller, Model und View
- Forum mit verschachtelten Baum-Komentaren
- Vollständiges Kommentar-CRUD (inkl. Thread-Löschung und Rechteprüfung)
- Nutzerrollen: Gast, Nutzer, Admin
- Benutzerverwaltung und Zitatpflege durch Admins
- Registrierungs- und Login-Mechanismen mit Sessions
- Schutz vor XSS & SQL-Injection (Prepared Statements, Escaping)
- Serverseitige Validierung aller Eingaben
- Datenbankstruktur mit Foreign Keys, CASCADE/SET NULL für Integrität
- Keine Abhängigkeit von externen Build-Tools: Läuft direkt auf XAMPP
- SQL-Dump und Demodaten enthalten

Weitere Details zur Architektur, Implementierung und zum Datenmodell folgen auf den nächsten Seiten.

1.1. Nutzerfluss nach Rolle

Das folgende Aktivitätsdiagramm zeigt typische Pfade durch die Anwendung. Die Navigation passt sich der Session an: Gäste sehen Links zu Login und Registrierung, eingeloggte Nutzer zu Profil und Logout, Admins zusätzlich Einträge in den Admin-Bereich.



1.2. Rollen und Berechtigungen

Drei Rollen steuern den Zugriff. Gäste sind nicht authentifiziert. Benutzer (`is_admin = 0`) dürfen eigene Kommentare erstellen, bearbeiten und löschen. Administratoren (`is_admin = 1`) verwalten zusätzlich Benutzer und Zitate und dürfen fremde Kommentare löschen, aber nicht bearbeiten.

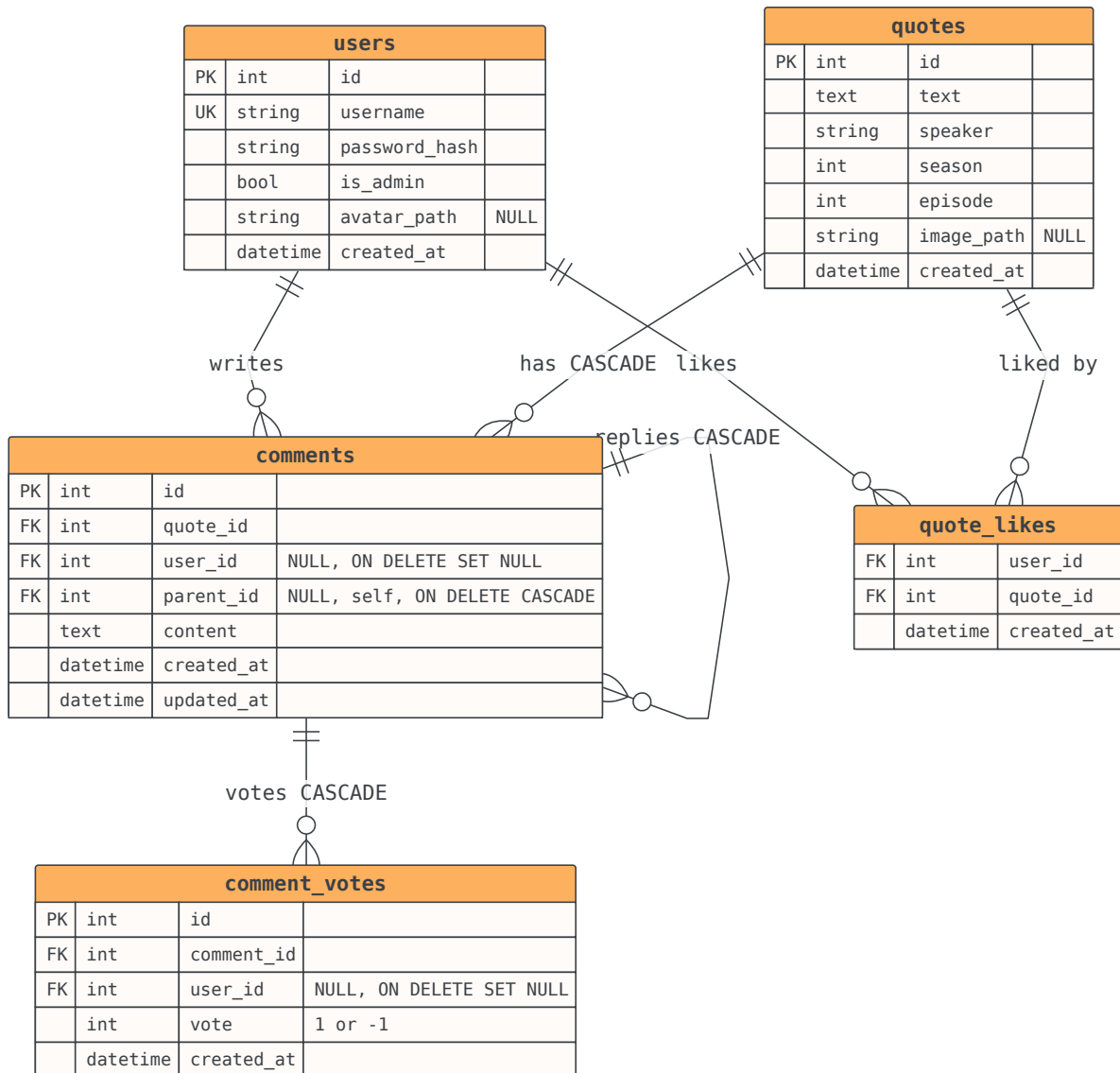
Aktion	Gast	User	Admin
Zitate und Kommentare lesen	✓	✓	✓
Registrierung und Login	✓	✓	✓
Kommentar schreiben, eigenes bearbeiten und löschen	-	✓	✓
Kommentare und Zitate bewerten bzw. liken	-	✓	✓
Fremden Kommentar löschen	-	-	✓
Fremden Kommentar bearbeiten	-	-	X
Zitate verwalten (CRUD)	-	-	✓
Benutzerverwaltung	-	-	✓

Wird ein Benutzer gelöscht, bleiben seine Kommentare in der Datenbank erhalten. Der Fremdschlüssel `user_id` wird per `ON DELETE SET NULL` auf `NULL` gesetzt und in der Benutzeroberfläche als graues `<deleted>` angezeigt. So bleibt der Diskussionsverlauf nachvollziehbar.

2. Datenmodell

2.1. ER-Diagramm

Das folgende Entity-Relationship-Diagramm wurde manuell modelliert und visualisiert die Entitäten sowie deren Beziehungen. Es entspricht dem SQL-Schema in `WEB4_PHP_TEAM7.sql`.



2.2. Normalisierung und Constraints

Die Datenbank folgt den Grundregeln des relationalen Designs. Jede Entität besitzt einen numerischen Primärschlüssel. Benutzernamen sind eindeutig. Fremdschlüssel sichern referenzielle Integrität: Kommentare und Likes werden mit dem zugehörigen Zitat gelöscht (`CASCADE`), Nutzerreferenzen in Kommentaren und Votes werden bei Nutzerlöschung auf `NULL` gesetzt (`SET NULL`). Antworten referenzieren den Elternkommentar über `parent_id`; beim Löschen eines Kommentars mit Kindern greift `ON DELETE CASCADE` auf die gesamte Unterdiskussion.

2.2.1. users

Spalte	Typ	Constraints
id	INT UNSIGNED	PK, AUTO_INCREMENT
username	VARCHAR(50)	NOT NULL, UNIQUE
password_hash	VARCHAR(255)	NOT NULL
is_admin	TINYINT(1)	NOT NULL, DEFAULT 0
avatar_path	VARCHAR(255)	NULL, Profilbild unter /uploads/avatars/
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

Passwörter werden ausschließlich als Hash gespeichert (`password_hash` via `password_hash()`). Klartextpasswörter existieren nicht in der Datenbank.

2.2.2. quotes

Spalte	Typ	Constraints
id	INT UNSIGNED	PK, AUTO_INCREMENT
text	TEXT	NOT NULL
speaker	VARCHAR(100)	NOT NULL
season / episode	TINYINT UNSIGNED	NULL, optional
image_path	VARCHAR(255)	NULL, Beitragsbild unter /uploads/quotes/
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

2.2.3. comments

Spalte	Typ	Constraints
id	INT UNSIGNED	PK, AUTO_INCREMENT
quote_id	INT UNSIGNED	FK → quotes.id, ON DELETE CASCADE
user_id	INT UNSIGNED	FK → users.id, NULL, ON DELETE SET NULL
parent_id	INT UNSIGNED	FK → comments.id, NULL, ON DELETE CASCADE
content	TEXT	NOT NULL
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
updated_at	DATETIME	NULL ON UPDATE

Kommentare sind die Haupt-CRUD-Ressource der Anwendung. `parent_id = NULL` kennzeichnet Top-Level-Kommentare, sonst handelt es sich um Thread-Antworten.

2.2.4. quote_likes

Spalte	Typ	Constraints
user_id	INT UNSIGNED	FK → users.id, ON DELETE CASCADE
quote_id	INT UNSIGNED	FK → quotes.id, ON DELETE CASCADE
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

Primärschlüssel ist das Paar `(user_id, quote_id)`. Pro Nutzer und Zitat ist höchstens ein Like möglich.

2.2.5. comment_votes

Spalte	Typ	Constraints
id	INT UNSIGNED	PK, AUTO_INCREMENT
comment_id	INT UNSIGNED	FK → comments.id, ON DELETE CASCADE
user_id	INT UNSIGNED	FK → users.id, NULL, ON DELETE SET NULL
vote	TINYINT	NOT NULL, Werte 1 oder -1
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

2.3. Testdaten

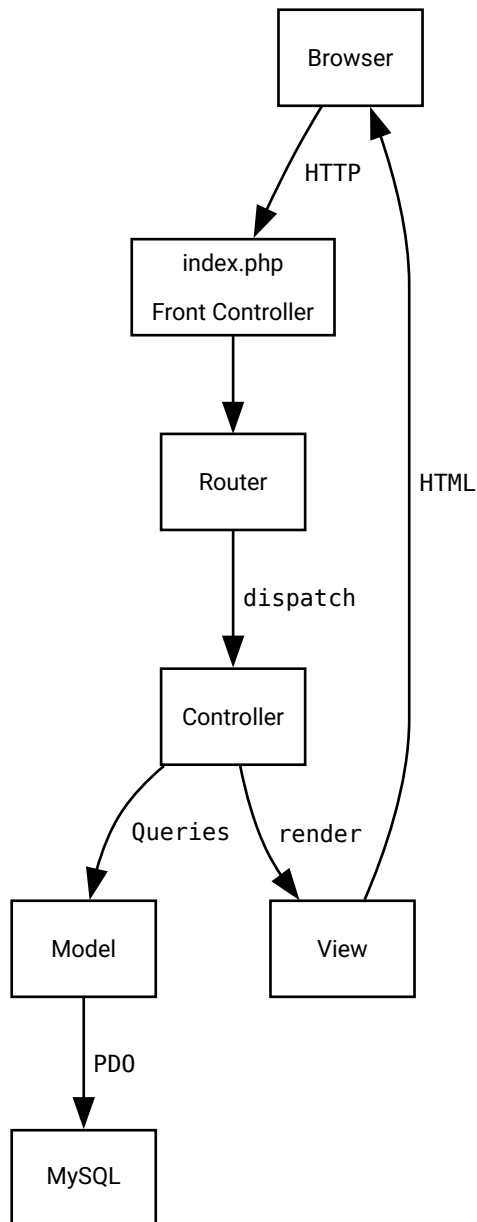
Der SQL-Dump `WEB4_PHP_TEAM7.sql` enthält das `CREATE DATABASE`-Statement, Tabellendefinitionen und repräsentative Testdaten. Nach dem Import stehen folgende Zugangsdaten zur Verfügung:

- Admin: `admin` / `admin`
- Testnutzer: `tyrion_fan`, `arya_fan` (Passwort jeweils `password123`)
- 12 Zitate, 17 Kommentare inklusive verschachtelter Antworten, Likes und Votes
- Bilder sind **nicht** inkludiert, können aber manuell hochgeladen werden

3. Architektur

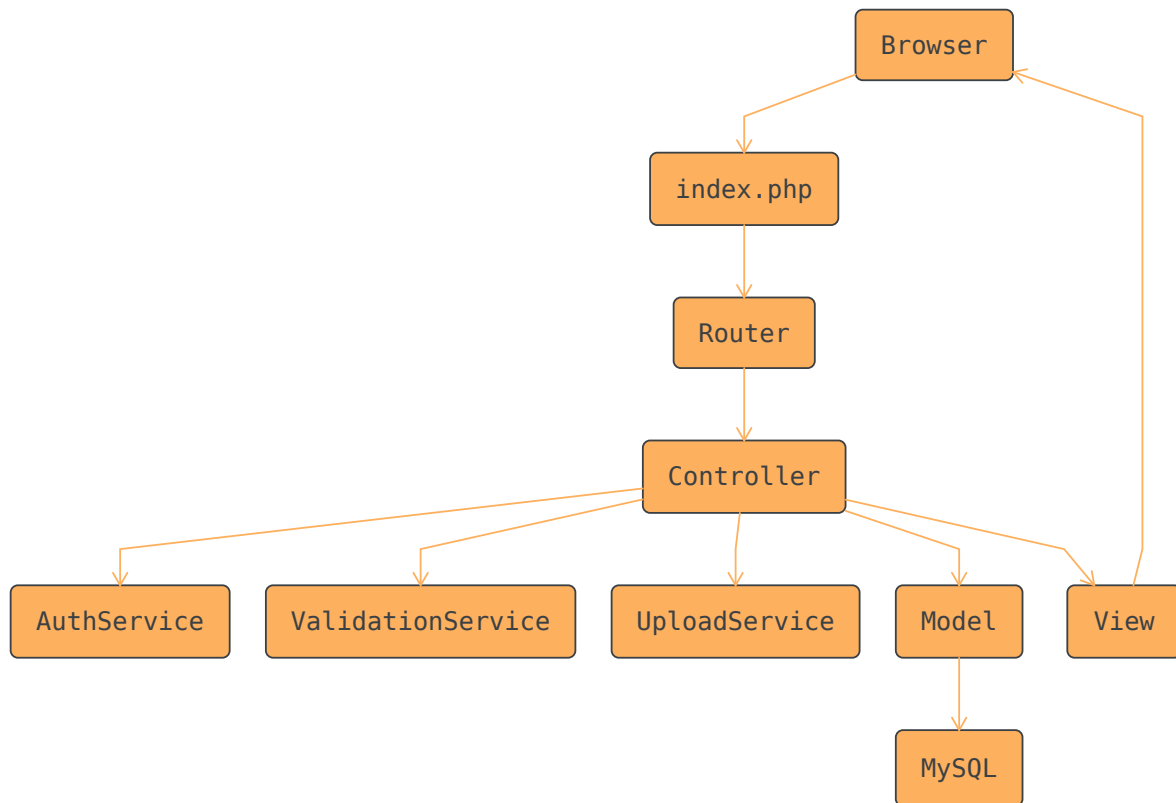
3.1. MVC

Die Projektangabe verlangt eine saubere Trennung der Anwendungsschichten. GoT Quotes implementiert klassisches MVC: Der Front Controller `index.php` leitet jede HTTP-Anfrage an den Router weiter. Controller orchestrieren Geschäftslogik und Berechtigungen, Models kapseln den PDO-Datenbankzugriff, Views rendern HTML mit escaped Ausgabe.

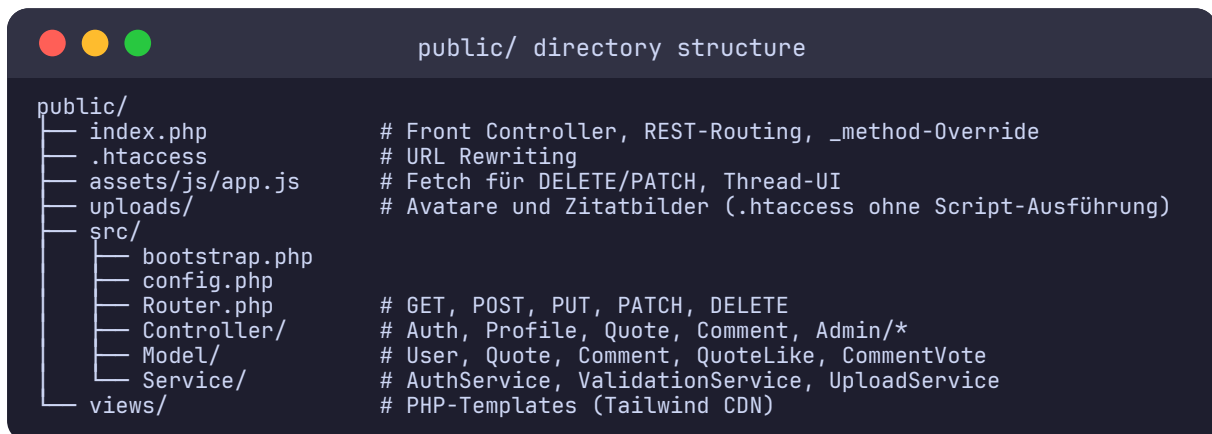


3.2. Komponentenübersicht

Innerhalb von `public/src/` sind die Verantwortlichkeiten klar getrennt. Services wie `AuthService`, `ValidationService` und `UploadService` enthalten querschnittliche Logik, die von mehreren Controllern genutzt wird.



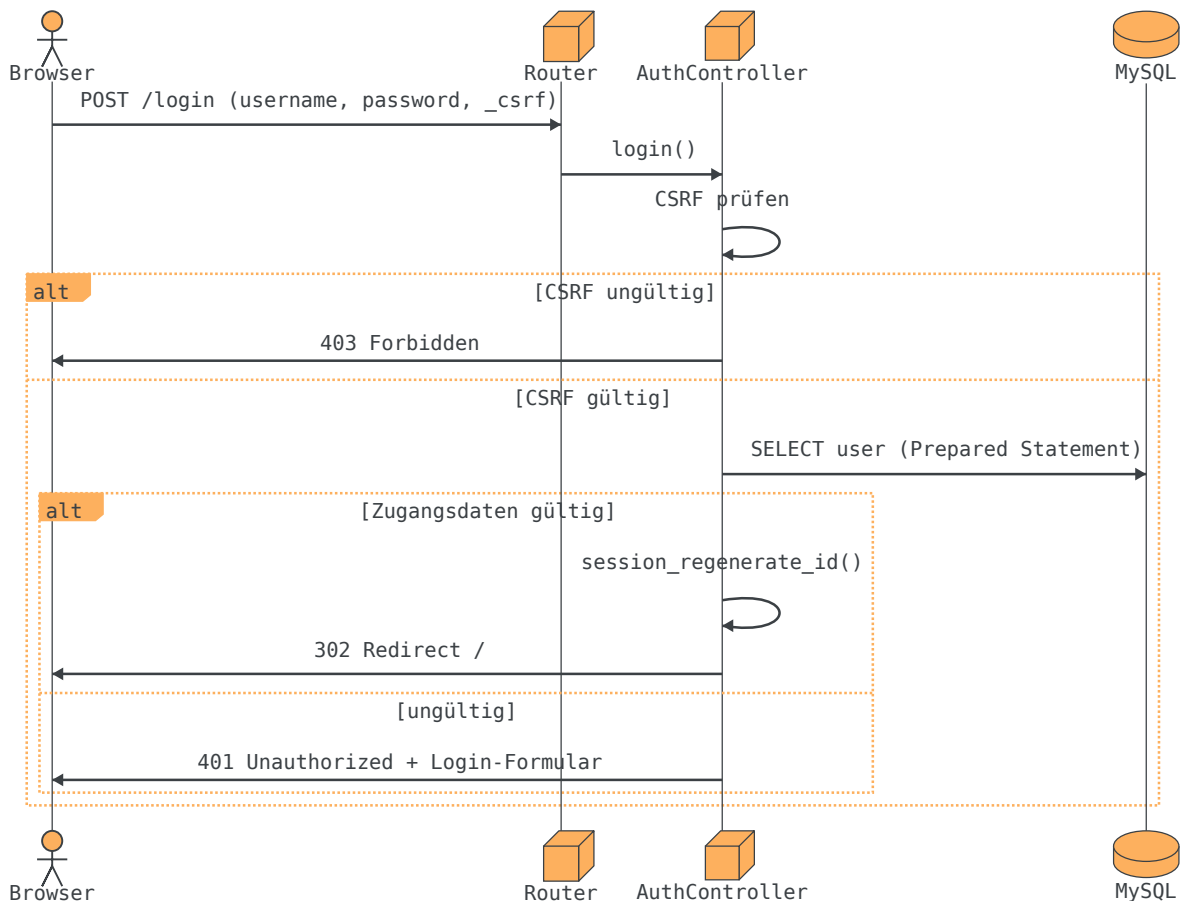
3.3. Verzeichnisstruktur



Jede Schicht kennt nur die darunterliegende: Views enthalten kein SQL, Models enthalten keine HTTP-Logik. Der Router mappt Pfade und HTTP-Methoden auf Controller-Methoden und extrahiert Platzhalter wie `{id}`.

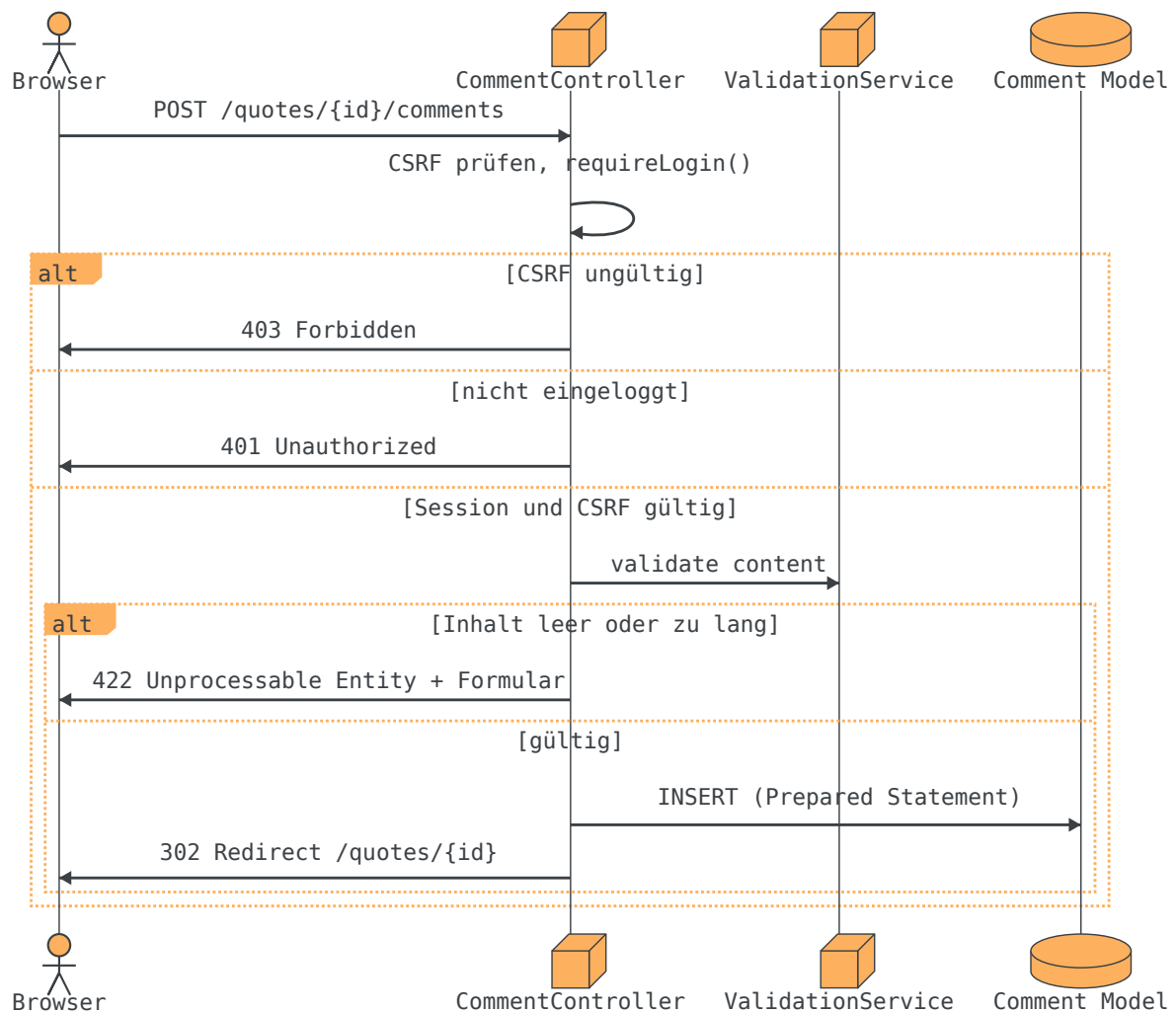
3.4. Ablauf: Login und Session

Authentifizierung erfolgt sessionbasiert. Nach erfolgreichem Login speichert PHP in `$_SESSION` die Felder `user_id`, `username`, `is_admin` und optional `avatar_path`. `session_regenerate_id()` verhindert Session-Fixation. Flash-Messages informieren einmalig über Erfolg oder Fehler nach Redirects.



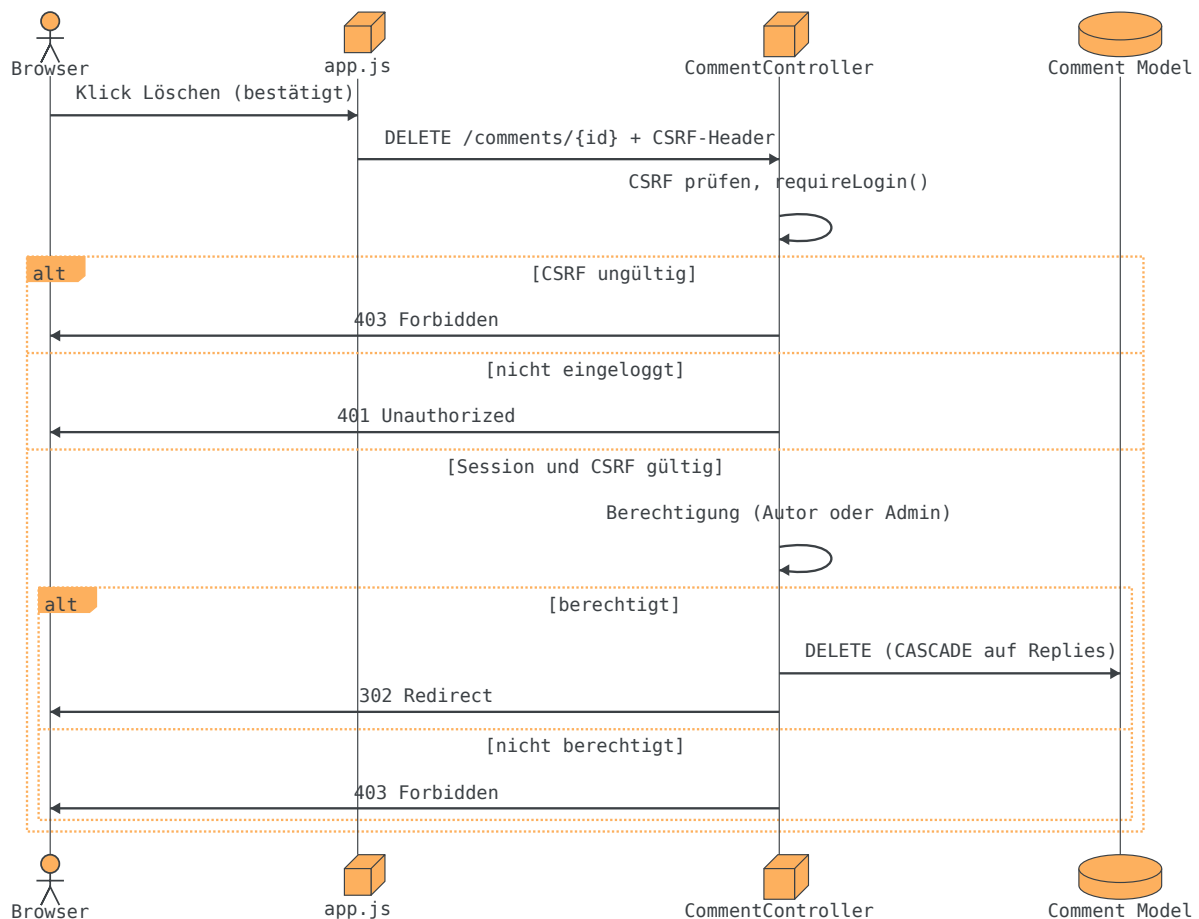
3.5. Ablauf: Kommentar anlegen (CRUD Create)

Das Erstellen eines Kommentars verdeutlicht die CRUD-Umsetzung für die Hauptressource. Der Controller prüft Login, CSRF und Validierung, bevor das Model den Datensatz einfügt. Fehlerhafte Eingaben liefern semantisch passende Statuscodes (401, 403, 422).



3.6. Ablauf: Kommentar löschen per REST

Löschungen nutzen die HTTP-Methode DELETE und werden per JavaScript Fetch ausgelöst, da HTML-Formulare DELETE nicht nativ unterstützen. Der CSRF-Token wird als Header `x-CSRF-Token` mitgesendet.



3.7. REST-Routing

Die Anwendung verwendet ressourcenorientierte URLs und semantisch korrekte HTTP-Methoden. Es gibt keine Aktions-URLs wie `/comments/delete`. Browser-Formulare ohne Fetch nutzen bei PUT/PATCH/DELETE das versteckte Feld `_method`; `index.php` setzt daraus die effektive Request-Methode.

Methode	Pfad	Aktion
GET	/	Zitate-Feed mit <code>?sort=new top trending</code>
GET	/quotes/{id}	Zitat-Detail mit Kommentar-Thread
POST/DELETE	/quotes/{id}/likes	Zitat liken bzw. Like entfernen
POST/DELETE	/comments/{id}/votes	Kommentar bewerten
POST	/quotes/{id}/comments	Top-Level-Kommentar erstellen
POST	/comments/{id}/replies	Thread-Antwort erstellen
PUT	/comments/{id}	Eigenen Kommentar bearbeiten
DELETE	/comments/{id}	Kommentar löschen (Autor oder Admin)
GET	/register	Registrierungsformular
POST	/register	Neuen Benutzer anlegen

Methode	Pfad	Aktion
GET/POST	/login	Login-Formular bzw. Authentifizierung
POST	/logout	Session beenden
GET	/users/{id}	Öffentliches Profil
GET/PATCH/DELETE	/admin/users ...	Benutzerverwaltung
GET/POST/PUT/DELETE	/admin/quotes ...	Zitat-CRUD für Admins

4. Backend und Sicherheit

4.1. PDO und Prepared Statements

Sämtliche Datenbankzugriffe laufen über PDO mit Prepared Statements. User-Input wird nie per String-Konkatenation in SQL eingebettet. Die Model-Klassen kapseln alle Queries und geben gebundene Parameter an PDO weiter. Damit ist die Anwendung gegen SQL-Injection abgesichert.

4.2. Validierung

Jeder textuelle Input aus Formularen oder Uploads wird serverseitig geprüft. Der `ValidationService` setzt unter anderem folgende Regeln durch:

Feld	Regeln
username	3 bis 50 Zeichen, nur <code>[a-zA-Z0-9_]</code> , eindeutig
password	mindestens 8 Zeichen
comment.content	1 bis 1000 Zeichen, nicht leer nach <code>trim()</code>
quote.text	1 bis 2000 Zeichen
quote.speaker	1 bis 100 Zeichen
Bild-Uploads	optional, max. 2 MB, JPEG/PNG/WebP, MIME-Check via <code>finfo</code>

Fehlerhafte Eingaben führen nicht zu stillschweigendem Verwerfen: Das Formular wird erneut angezeigt, Fehlermeldungen erscheinen am betroffenen Feld oder als Flash-Message. Auch in den automatisierten Tests (`cmtEmptyRejected`, `authLoginFail`) werden fehlerhafte Eingaben explizit geprüft.

4.3. Schutzmaßnahmen im Überblick

Bedrohung	Maßnahme
SQL Injection	PDO Prepared Statements für alle Queries mit User-Input
XSS	<code>htmlspecialchars()</code> bei jeder Ausgabe usergenerierter Inhalte
Session Fixation	<code>session_regenerate_id()</code> nach Login

Bedrohung	Maßnahme
CSRF	Token in POST-Formularen und Fetch-Header <code>X-CSRF-Token</code>
IDOR	Bearbeitung nur für Kommentar-Autor; Admin darf fremde Kommentare nur löschen
Passwörter	<code>password_hash()</code> / <code>password_verify()</code> , <code>PASSWORD_DEFAULT</code>
Uploads	MIME-Check, Größenlimit, zufällige Dateinamen, <code>.htaccess</code> ohne Script-Ausführung

5. Frontend und Benutzeroberfläche

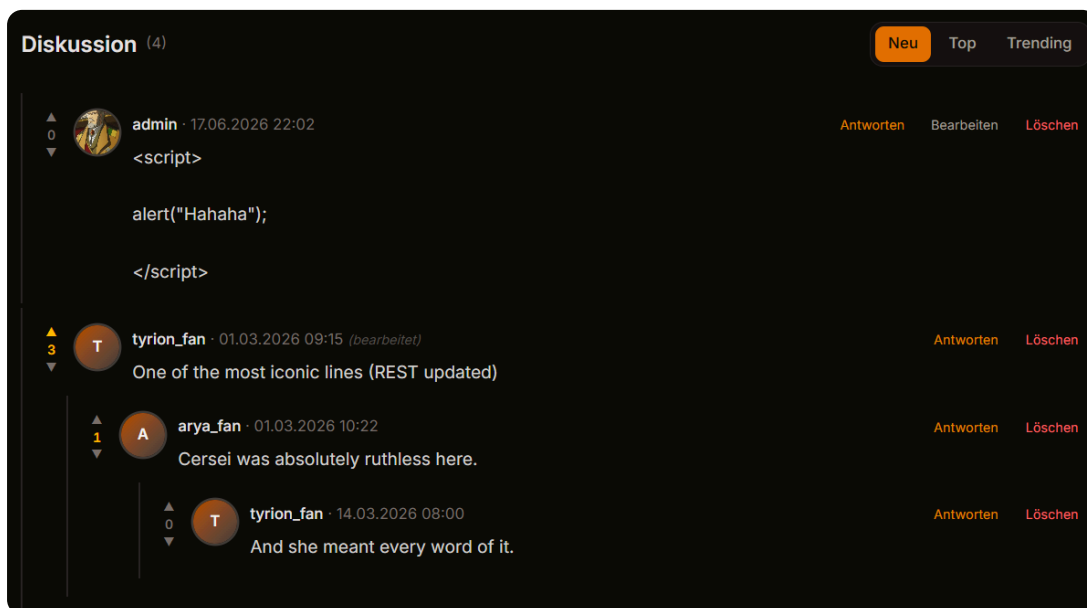
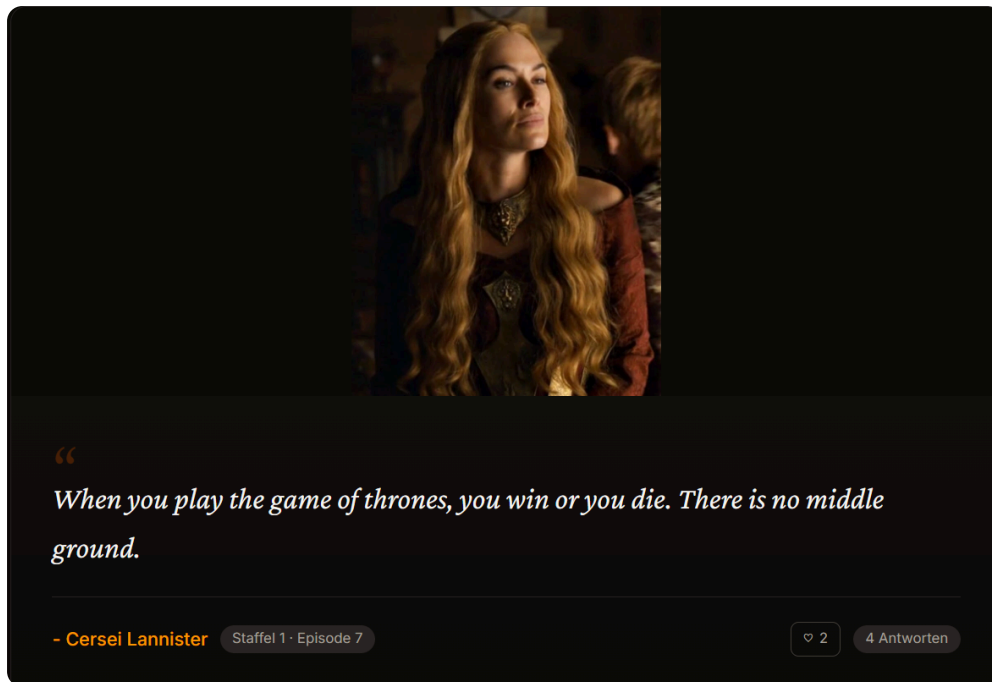
5.1. HTML, CSS und JavaScript

Die Oberfläche besteht aus validem HTML5. Formatierungen erfolgen ausschließlich über CSS, konkret [Tailwind CSS 4](#) per CDN im Layout-Template. Die Navigation führt Gäste zu Feed, Login und Registrierung, eingeloggte Nutzer zusätzlich zu Profil und Logout, Administratoren zu den Admin-Bereichen Benutzer und Zitate.

JavaScript ist bewusst auf Transport- und Komfortfunktionen beschränkt. Die Datei `public/assets/js/app.js` sendet per Fetch API DELETE- und PATCH-Anfragen, übergibt CSRF-Token aus dem Meta-Tag und blendet Antwort-Formulare im Thread ein. Validierung, Berechtigungsprüfung und Escaping bleiben vollständig serverseitig, wie von der Projektangabe gefordert.

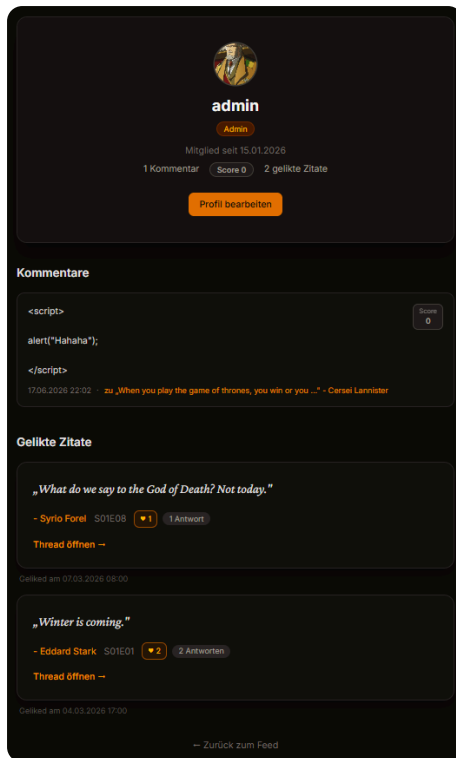
5.2. Foren- und Thread-Darstellung

Zitate erscheinen im Feed als Karten mit Ausschnitt, Sprecher, optional Thumbnail, Kommentar- und Like-Zähler. Auf der Detailseite zeigt ein Hero-Bild (falls vorhanden) das vollständige Zitat. Kommentare werden rekursiv in `comment-tree.php` gerendert: Avatare links, Einrückung und Verbindungslinien visualisieren die Thread-Struktur. Sortierung ist sowohl im Feed (`?sort=`) als auch bei Kommentaren (`?csort=`) wählbar.



5.3. Public Profile

Öffentliche Profile zeigen den Namen, Avatar, Kommentar- und Like-Statistik. Sie sind bei allen Anwendern zugänglich und sind von Kommentaren aus navigierbar.



5.4. Admin-Bereich

Die Benutzerverwaltung zeigt tabellarisch alle Benutzer mit ID, Namen, Rolle, Registrierungsdatum und Aktionen. Hier ist es möglich, Benutzer zu löschen oder die Admin-Rolle zu ändern.

Benutzerverwaltung				
ID	BENUTZER	ROLLE	REGISTRIERT	AKTIONEN
1	admin	Admin	15.01.2026	(Du)
2	tyrion_fan	User	01.02.2026	Zum Admin machen Löschen
3	arya_fan	User	10.02.2026	Zum Admin machen Löschen

← Zurück zur Startseite

Die Zitatverwaltung ermöglicht das CRUD für Zitate. Auch hier gibt es eine übersichtliche Darstellung mittels Tabelle, die ID, Text, Sprecher, Thumbnail und Aktionen anzeigt. Alle CRUD-Operationen öffnen ein eigenes Formular, um das Zitat zu erstellen, zu bearbeiten oder anzusehen.

Zitat-Verwaltung

Neues Zitat

Zur Startseite

ID	ZITAT	SPRECHER	BILD	AKTIONEN		
1	When you play the game of thrones, you win or y...	Cersei Lannister	✓	Ansehen	Bearbeiten	Löschen
2	The night is dark and full of terrors.	Melisandre	-	Ansehen	Bearbeiten	Löschen
3	A Lannister always pays his debts.	Tyrion Lannister	-	Ansehen	Bearbeiten	Löschen
4	Winter is coming.	Eddard Stark	-	Ansehen	Bearbeiten	Löschen
5	Hold the door!	Hodor	-	Ansehen	Bearbeiten	Löschen
6	Any man who must say "I am the king" is no true	Tywin Lannister	-	Ansehen	Bearbeiten	Löschen

6. Testfälle

Die Projektangabe verlangt ausführliche Tests der Anwendung, einschließlich fehlerhafter Eingaben. Wir unterscheiden automatisierte REST-Tests und manuelle UI-Tests in Google Chrome unter XAMPP.

6.1. Automatisierter REST-Testlauf

Die REST-Schnittstelle wird mit [httpYac](#) getestet. Jede `.rest`-Datei beschreibt HTTP-Anfragen mit Assertions (`?? status = 200`, `?? body includes ...`). Der Lauf erzeugt einen JUnit-Report.

Schritt	Befehl bzw. Pfad
Tests ausführen	<code>httpyac send "tests/rest/*.rest" --all -e dev</code>
Testdateien	<code>tests/rest/*</code>
Voraussetzung	App unter <code>http://127.0.0.1:80</code> , SQL-Dump importiert

Die Dateien folgen einem einheitlichen Namensschema und Fehlerfälle sind darin enthalten, etwa Login mit falschem Passwort, leerer Kommentar, Zugriff auf Admin ohne Rolle und DELETE ohne CSRF.

Die folgenden Tabellen wurden aus dem JUnit-Report generiert. Spalte **Prüfung** stammt aus der Assertion in der `.rest`-Datei, **Beobachtet** ist der tatsächliche Wert.

6.1.1. Übersicht REST-Tests

Testsuite	Tests	Fehler
Total	46	0
tests/rest/01-public.rest	11	0
tests/rest/02-auth.rest	11	0
tests/rest/03-comments.rest	11	0
tests/rest/04-admin.rest	7	0
tests/rest/05-forum.rest	6	0

6.1.2. Erwartete und beobachtete Werte

Request	Prüfung	Beobachtet	Status
tests/rest/01-public.rest			
pubFeed	status == 200	200	OK
pubFeed	body contains Zitate		OK
pubQuoteDetail	status == 200	200	OK
pubQuoteDetail	body contains Cersei Lannister		OK
pubQuote404	status == 404	404	OK
pubProfile	status == 200	200	OK
pubProfile	body contains tyrion_fan		OK
pubSortTop	status == 200	200	OK
pubSortTop	body contains sort=top		OK
pubCommentSort	status == 200	200	OK
pubAdminForbidden	status == 403	403	OK
tests/rest/02-auth.rest			
authLoginForm	status == 200	200	OK
authLoginForm	body contains Einloggen		OK
authLoginOk	status == 302	302	OK
authLoginOk	location == /	/	OK
authSessionCheck	status == 200	200	OK
authSessionCheck	body contains tyrion_fan		OK
authLogout	status == 302	302	OK
authProfileRedirect	status == 302	302	OK
authProfileRedirect	location == /login	/login	OK
authLoginFail	status == 401	401	OK
authLoginFail	body contains ungültig		OK
tests/rest/03-comments.rest			
cmtCreate	status == 302	302	OK
cmtShow	status == 200	200	OK
cmtShow	body contains Automatisierter REST-Test		OK
cmtForbiddenEdit	status == 403	403	OK
cmtEditForm	status == 200	200	OK
cmtEditForm	body contains Kommentar bearbeiten		OK
cmtUpdate	status == 302	302	OK
cmtUpdated	status == 200	200	OK
cmtUpdated	body contains REST updated		OK
cmtEmptyRejected	status == 422	422	OK
cmtEmptyRejected	body contains darf nicht leer		OK

Request	Prüfung	Beobachtet	Status
tests/rest/04-admin.rest			
admUserList	status == 200	200	OK
admUserList	body contains Benutzer		OK
admQuoteList	status == 200	200	OK
admQuoteList	body contains Zitate		OK
admQuoteNewForm	status == 200	200	OK
admQuoteNewForm	body contains Anlegen		OK
admUserForbidden	status == 403	403	OK
tests/rest/05-forum.rest			
forumLike	status == 302	302	OK
forumUnlike	status == 302	302	OK
forumUpvote	status == 302	302	OK
forumRemoveVote	status == 302	302	OK
forumDeleteNoCsrf	status == 403	403	OK
forumBadLike	status == 404	404	OK

6.2. Manuelle UI-Tests

Zusätzlich zu den automatisierten Tests wurden Szenarien manuell geprüft. Dazu gehören Registrierung und Validierung, CRUD an Kommentaren, Admin-Funktionen, Foren-Interaktionen (Likes, Votes, Sortierung), Bild-Uploads, responsives Layout und Sicherheitsfälle wie XSS-Escaping und IDOR-Schutz.

6.2.1. Übersicht manuelle Tests

Testsuite	Tests	Fehler
Total	32	0
Authentifizierung	6	0
Zitate & Kommentare	9	0
Administration	8	0
Forum & Interaktion	6	0
Sicherheit	3	0

6.2.2. Detailergebnisse manuelle Tests

Testname	Status	Zeit
Authentifizierung		
Registrierung gültig → Erfolg, Redirect Login	✓ Erfolgreich	manuell
Duplicate Username → Fehler	✓ Erfolgreich	manuell
Passwort < 8 Zeichen → Fehler	✓ Erfolgreich	manuell
Login korrekt → Session aktiv	✓ Erfolgreich	manuell

Testname	Status	Zeit
Authentifizierung		
Login falsch → generische Fehlermeldung	✓ Erfolgreich	manuell
Logout → Session beendet	✓ Erfolgreich	manuell
Zitate & Kommentare		
Gast sieht Liste und Detail	✓ Erfolgreich	manuell
Gast sieht Kommentare, kein Formular	✓ Erfolgreich	manuell
User erstellt Kommentar mit Username	✓ Erfolgreich	manuell
User bearbeitet eigenen Kommentar	✓ Erfolgreich	manuell
User löscht eigenen Kommentar	✓ Erfolgreich	manuell
Fremdes Edit → 403	✓ Erfolgreich	manuell
Fremdes Delete → 403	✓ Erfolgreich	manuell
Leerer Kommentar → Validierungsfehler	✓ Erfolgreich	manuell
XSS-Payload → escaped in Ausgabe	✓ Erfolgreich	manuell
Administration		
Admin löscht fremden Kommentar	✓ Erfolgreich	manuell
Admin kann fremden Kommentar nicht bearbeiten	✓ Erfolgreich	manuell
Admin togglet User-Rolle	✓ Erfolgreich	manuell
Admin löscht User → Kommentare zeigen <deleted>	✓ Erfolgreich	manuell
Nicht-Admin auf /admin → 403	✓ Erfolgreich	manuell
Letzter Admin nicht degradierbar	✓ Erfolgreich	manuell
Admin CRUD Zitate	✓ Erfolgreich	manuell
Admin kann eigene Rolle nicht entziehen	✓ Erfolgreich	manuell
Forum & Interaktion		
Feed-Sortierung top/trending	✓ Erfolgreich	manuell
Zitat liken und entliken	✓ Erfolgreich	manuell
Kommentar up-/downvoten	✓ Erfolgreich	manuell
Kommentar-Sortierung auf Detailseite	✓ Erfolgreich	manuell
Öffentliches Profil mit Kommentaren und Likes	✓ Erfolgreich	manuell
Thread-Antworten und CASCADE-Löschung	✓ Erfolgreich	manuell
Sicherheit		
SQL-Injection Login → kein Bypass	✓ Erfolgreich	manuell
Direktaufruf fremdes Edit → 403	✓ Erfolgreich	manuell
Ungültige Quote-ID → 404	✓ Erfolgreich	manuell

7. Installation und Abgabe

7.1. Voraussetzungen

Die Anwendung ist für eine Standard-XAMPP-Installation unter Windows vorgesehen (Apache, PHP 8.x, MySQL/MariaDB, Google Chrome als Testbrowser). Es ist kein Composer, npm oder Build-Schritt nötig: Das entpackte ZIP genügt zum Betrieb.

7.2. Installationsschritte

1. ZIP-Archiv `WEB4_PHP_TEAM7.zip` entpacken
2. In phpMyAdmin den SQL-Dump `WEB4_PHP_TEAM7.sql` importieren (enthält `CREATE DATABASE`)
3. Apache DocumentRoot auf `public/` setzen
4. Anwendung unter `http://localhost/` aufrufen
5. Optional mit `admin / admin` einloggen und Admin-Bereich prüfen

7.3. Datenbankzugang

Parameter	Wert
Datenbankname	<code>team_7</code>
Benutzername	<code>fh_webphp</code>
Passwort	<code>fh_webphp</code>
SQL-Dump	<code>WEB4_PHP_TEAM7.sql</code>

Die Konfiguration liegt in `public/src/config.php` bzw. optional `config.local.php`. Vor der Abgabe wurde geprüft, dass der Dump ohne Fehler importiert werden kann und die Anwendung danach unter der dokumentierten Start-URL erreichbar ist.

7.4. Abgabeformat

Gemäß Projektangabe wird ein ZIP-Archiv im Schema `WEB4_PHP_TEAM7.zip` abgegeben. Enthalten sind der Anwendungscode unter `public/`, die Dokumentation als PDF unter `doc/`, der SQL-Dump und die REST-Testdateien unter `tests/rest/`.